

Since you already have the step count working over I2C on your MIMXRT1064-EVK, we can use the LSM303DLHC's built-in magnetometer to find the heading angle (compass direction).

By tracking the heading, the script checks if the sensor has turned right relative to your starting direction. If it detects a right turn, it accumulates the distance using a standard stride length calculation based on the steps taken during that turn.

Here is the self-contained MicroPython script ready to paste directly into Thonny:

Python

```
import machine
```

```
import time
```

```
import math
```

```
# --- Configuration ---
```

```
# Update these pin numbers if your I2C1 setup uses different pins
```

```
I2C_BUS = 1
```

```
SCL_PIN = 25
```

```
SDA_PIN = 24
```

```
# Stride length for distance tracking (approx. 0.75 meters per step)
```

```
STRIDE_LENGTH = 0.75
```

```
# Sensor I2C Addresses
```

```
ACCEL_ADDR = 0x19
```

```
MAG_ADDR = 0x1E
```

```
# --- Initialize I2C ---
```

```
i2c = machine.I2C(I2C_BUS, scl=machine.Pin(SCL_PIN), sda=machine.Pin(SDA_PIN), freq=400000)
```

```
def init_sensor():
```

```
    # Wake up Accelerometer (Normal power mode, 50 Hz, all axes enabled)
```

```
    i2c.writeto_mem(ACCEL_ADDR, 0x20, b'\x47')
```

```
    # Wake up Magnetometer (Continuous conversion mode)
```

```
    i2c.writeto_mem(MAG_ADDR, 0x02, b'\x00')
```

```

def read_mag():
    # Read X, Z, Y registers from magnetometer (LSM303DLHC orders them X, Z, Y)
    data = i2c.readfrom_mem(MAG_ADDR, 0x03, 6)

    # Convert to 16-bit signed integers
    x = (data[0] << 8) | data[1]
    z = (data[2] << 8) | data[3]
    y = (data[4] << 8) | data[5]

    if x > 32767: x -= 65536
    if y > 32767: y -= 65536

    return x, y

def get_heading():
    x, y = read_mag()

    # Calculate heading in degrees relative to the sensor's X axis
    heading = math.atan2(y, x) * 180 / math.pi
    if heading < 0:
        heading += 360
    return heading

# Simulated step counter function
# Replace the return value logic below with your working step detection variable/code
def check_for_step():
    # Insert your functional step validation code here
    time.sleep(0.2)
    return False

# --- Main Program Execution ---

```

```
init_sensor()

print("Calibrating baseline direction... Keep sensor steady.")

time.sleep(1)


initial_heading = get_heading()

right_turn_distance = 0.0

step_count_during_right_turn = 0


print(f"Starting Baseline Heading: {initial_heading:.2f}°")

print("Tracking distance exclusively during right turns...")


while True:

    current_heading = get_heading()


    # Calculate angular difference relative to start
    heading_diff = (current_heading - initial_heading) % 360


    # Identify if the heading falls into a right turn window (e.g., 30° to 150° clockwise)
    if 30 <= heading_diff <= 150:

        is_turning_right = True

        status = "TURNING RIGHT"

    else:

        is_turning_right = False

        status = "FORWARD/OTHER"


    # Process steps if detected
    if check_for_step():

        if is_turning_right:

            step_count_during_right_turn += 1

            right_turn_distance = step_count_during_right_turn * STRIDE_LENGTH
```

```
print(f"Status: {status} | Heading: {current_heading:.1f}° | Right Turn Distance:
{right_turn_distance:.2f} meters")

time.sleep(0.1)
```

How the logic works:

1. **Compass Heading:** The script reads the raw X and Y magnetic values from the 0x1E register address space, using $\text{math.atan2}(y, x)$ to resolve a compass heading between 0° and 360° .
2. **Right Turn Evaluation:** It establishes your initial orientation at startup. A relative change between 30° and 150° clockwise is designated as a right turn zone.
3. **Distance Calculation:** While the sensor remains inside that right-hand angular window, any steps generated increments your tracking distance ($\text{Steps} \times \text{Stride Length}$). When moving straight or turning left, the distance calculation pauses.

To avoid using software bit-banging (SoftI2C), we will use the hardware I2C peripheral blocks built into the RT1064 microcontroller.

On the NXP MIMXRT1064-EVK board using MicroPython, the hardware I2C peripherals are instantiated by passing the specific bus ID parameter directly to `machine.I2C()`.

Here is the updated, self-contained code for Thonny that forces the engine to run strictly over the hardware I2C bus:

Python

```
import machine

import time

import math
```

```
# --- Configuration ---
```

```
# Bus ID 1 initializes hardware I2C (LPI2C peripheral block)
```

```
I2C_BUS_ID = 1
```

```
SCL_PIN = 25
```

```
SDA_PIN = 24
```

```
# Stride length conversion factor (meters per step)
```

```
STRIDE_LENGTH = 0.75
```

```
# LSM303DLHC Hardware Slave Addresses
```

```
ACCEL_ADDR = 0x19
```

```
MAG_ADDR = 0x1E
```

```
# --- Initialize Hardware I2C ---
```

```
# Passing the integer ID without keyword flags initializes pure Hardware I2C
```

```
i2c = machine.I2C(I2C_BUS_ID, scl=machine.Pin(SCL_PIN), sda=machine.Pin(SDA_PIN), freq=400000)
```

```
def init_sensor():
```

```
    # Wake up Accelerometer (Normal power mode, 50 Hz, all axes enabled)
```

```
    i2c.writeto_mem(ACCEL_ADDR, 0x20, b'\x47')
```

```
    # Wake up Magnetometer (Continuous conversion mode)
```

```
    i2c.writeto_mem(MAG_ADDR, 0x02, b'\x00')
```

```
def read_mag():
```

```
    # Read 6 bytes starting at register 0x03 (OUT_X_H_M)
```

```
    data = i2c.readfrom_mem(MAG_ADDR, 0x03, 6)
```

```
    # Reconstruct sign-extended 16-bit values from registers
```

```
    x = (data[0] << 8) | data[1]
```

```
    z = (data[2] << 8) | data[3]
```

```
    y = (data[4] << 8) | data[5]
```

```
    if x > 32767: x -= 65536
```

```
    if y > 32767: y -= 65536
```

```
    return x, y
```

```
def get_heading():
```

```
    x, y = read_mag()
```

```
    # Compute plane vector angle to find magnetic heading
```

```
    heading = math.atan2(y, x) * 180 / math.pi
```

```

    if heading < 0:
        heading += 360
    return heading

# --- Placeholder Step Integration ---
# Inject your functioning step verification variables here
def check_for_step():
    time.sleep(0.1)
    return False

# --- Main Program Execution ---
init_sensor()
print("Hardware I2C Initialized. Calibrating baseline direction...")
time.sleep(1)

initial_heading = get_heading()
right_turn_distance = 0.0
step_count_during_right_turn = 0

print(f"Initial Starting Heading: {initial_heading:.2f}°")

while True:
    current_heading = get_heading()

    # Calculate angular distance relative to startup angle
    heading_diff = (current_heading - initial_heading) % 360

    # Evaluate if current direction sits within a right turn window (e.g., 30° to 150°)
    if 30 <= heading_diff <= 150:
        is_turning_right = True
        status = "TURNING RIGHT"

```

```

else:

    is_turning_right = False

    status = "FORWARD/OTHER"

# Accumulate data exclusively while right turns are maintained
if check_for_step():
    if is_turning_right:
        step_count_during_right_turn += 1
        right_turn_distance = step_count_during_right_turn * STRIDE_LENGTH

    print(f"[{status}] Heading: {current_heading:.1f}° | Right Turn Distance: {right_turn_distance:.2f} m")

```

This updated code tracks **everything**: total steps, total distance traveled overall, and breaks down the specific distances covered during forward tracking versus when making a right turn.

It uses pure hardware I2C without soft bit-banging.

Python

```
import machine
```

```
import time
```

```
import math
```

```
# --- Configuration ---
```

```
I2C_BUS_ID = 1
```

```
SCL_PIN = 25
```

```
SDA_PIN = 24
```

```
# Stride length conversion factor (meters per step)
```

```
STRIDE_LENGTH = 0.75
```

```
# LSM303DLHC Hardware Slave Addresses
```

```
ACCEL_ADDR = 0x19
```

```
MAG_ADDR = 0x1E
```

```
# --- Initialize Hardware I2C ---
```

```
i2c = machine.I2C(I2C_BUS_ID, scl=machine.Pin(SCL_PIN), sda=machine.Pin(SDA_PIN), freq=400000)
```

```
def init_sensor():
```

```
    # Wake up Accelerometer (Normal power mode, 50 Hz, all axes enabled)
```

```
    i2c.writeto_mem(ACCEL_ADDR, 0x20, b'\x47')
```

```
    # Wake up Magnetometer (Continuous conversion mode)
```

```
    i2c.writeto_mem(MAG_ADDR, 0x02, b'\x00')
```

```
def read_mag():
```

```
    # Read 6 bytes starting at register 0x03 (OUT_X_H_M)
```

```
    data = i2c.readfrom_mem(MAG_ADDR, 0x03, 6)
```

```
    # Reconstruct sign-extended 16-bit values from registers
```

```
    x = (data[0] << 8) | data[1]
```

```
    z = (data[2] << 8) | data[3]
```

```
    y = (data[4] << 8) | data[5]
```

```
    if x > 32767: x -= 65536
```

```
    if y > 32767: y -= 65536
```

```
    return x, y
```

```

def get_heading():
    x, y = read_mag()

    # Compute plane vector angle to find magnetic heading
    heading = math.atan2(y, x) * 180 / math.pi

    if heading < 0:
        heading += 360

    return heading

# --- Placeholder Step Integration ---

# Replace this function's contents with your working step detection flag logic
def check_for_step():
    time.sleep(0.1)

    return False

# --- Main Program Execution ---

init_sensor()

print("Hardware I2C Initialized. Calibrating baseline direction...")

time.sleep(1)

initial_heading = get_heading()

# Distance Accumulator Metrics

total_steps = 0

total_distance = 0.0

right_turn_distance = 0.0

forward_distance = 0.0

print(f"Initial Starting Heading: {initial_heading:.2f}°")

print("Tracking all motion components concurrently...")

```

```

while True:

    current_heading = get_heading()

    # Calculate angular distance relative to startup angle
    heading_diff = (current_heading - initial_heading) % 360

    # Categorize steering state based on current heading quadrant
    if 30 <= heading_diff <= 150:

        status = "TURNING RIGHT"

        movement_state = "right"

    elif 330 >= heading_diff >= 210:

        status = "TURNING LEFT"

        movement_state = "left"

    else:

        status = "MOVING FORWARD"

        movement_state = "forward"

    # Track every step and segment the distances
    if check_for_step():

        total_steps += 1

        total_distance = total_steps * STRIDE_LENGTH

        if movement_state == "right":

            right_turn_distance += STRIDE_LENGTH

        elif movement_state == "forward":

            forward_distance += STRIDE_LENGTH

    # Clear visual status layout for tracking printouts
    print(f"Direction: {status:<15} | Angle: {current_heading:5.1f}°")

    print(f" -> Total Steps: {total_steps} | Total Dist: {total_distance:.2f}m")

    print(f" -> Forward Dist: {forward_distance:.2f}m | Right Turn Dist: {right_turn_distance:.2f}m")

```

```
print("-" * 65)
```